Construcción de un Pipeline CI/CD para el Sistema de Gestión de Restaurantes Red Velvet

Andres Felipe Jimenez Camacho

2026

Introducción

Contexto del Sistema de Restaurante

Red Velvet es un sistema web unificado de gestión de restaurantes diseñado para centralizar y automatizar las operaciones diarias de un establecimiento gastronómico. La plataforma integra módulos funcionales que abarcan desde la gestión de pedidos y reservas en línea hasta la facturación, el control de inventario, la administración de mesas y la personalización visual del sistema. La aplicación está construida sobre una arquitectura cliente-servidor moderna, con un backend desarrollado en Node.js con Express y una base de datos PostgreSQL alojada en Neon (serverless), mientras que el frontend es una Single Page Application (SPA) construida con React y Vite.

El sistema maneja cuatro roles de usuario estrictamente definidos mediante control de acceso basado en roles (RBAC): Master (superadministrador con acceso total a la configuración del sistema), Administrador (gestión de pedidos, caja, mesas y reservas), Mesero (creación de pedidos y gestión de mesas asignadas) y Cliente (exploración del menú, carrito de compras, checkout y seguimiento de pedidos). La autenticación y autorización se implementan mediante tokens JWT (JSON Web Tokens), garantizando que cada usuario solo acceda a los recursos y funcionalidades correspondientes a su nivel de privilegios.

El ecosistema de Red Velvet se encuentra actualmente desplegado en dos entornos de producción en la nube: el backend está alojado en Railway (`https://loyal-smile-production-2893.up.railway.app`), mientras que el frontend está desplegado en Vercel (`https://frontend-chi-five-62.vercel.app`), conectado directamente al repositorio de GitHub para despliegues automáticos.

Justificación de la Necesidad de un Pipeline Automatizado

En el ciclo de vida del desarrollo de software moderno, particularmente en sistemas transaccionales y de alta concurrencia como Red Velvet, la validación constante de los procesos de negocio constituye una actividad crítica. Este sistema gestiona operaciones en tiempo real que impactan directamente el flujo de ingresos y la experiencia del usuario, tales como la facturación, el control de inventario, la coordinación entre el personal de salón y cocina, y la pasarela de pedidos en línea.

Depender exclusivamente de pruebas manuales para validar estas operaciones genera un cuello de botella significativo. A medida que el código evoluciona y nuevas funcionalidades se integran, verificar manualmente cada actualización cruzándola con los distintos roles del sistema (Master, Administrador, Mesero y Cliente) resulta insostenible, costoso y altamente propenso al error humano. Un fallo no detectado en la lógica de la API o en la interfaz de usuario puede traducirse en pérdidas económicas directas, deterioro de la experiencia del cliente y daño a la reputación del establecimiento.

La implementación de un pipeline de Integración y Entrega Continua (CI/CD) se justifica bajo los siguientes pilares fundamentales:

1. **Detección temprana de errores**: Automatizar las validaciones de código, compilación y pruebas permite identificar defectos funcionales en etapas tempranas del desarrollo, reduciendo drásticamente el costo de corrección. Un error detectado durante la fase de integración

cuesta significativamente menos que uno encontrado en producción.

2. **Reducción de tiempos de regresión**: Lo que a un operador humano le tomaría horas en validaciones cruzadas (como iniciar sesión con diferentes roles, crear pedidos y verificar la integridad de los datos), una suite automatizada lo ejecuta de manera exhaustiva en menos de dos minutos. Esto libera recursos humanos para tareas de mayor valor, como pruebas exploratorias y mejora de la experiencia de usuario.
3. **Entregas frecuentes y confiables**: Un pipeline automatizado elimina la intervención manual en los procesos de construcción, prueba y despliegue, permitiendo que nuevas versiones del software lleguen a producción de forma rápida y segura. Esto se alinea con las prácticas de DevOps y entrega continua, donde los ciclos de retroalimentación cortos son esenciales.
4. **Trazabilidad y auditoría**: Cada ejecución del pipeline genera artefactos y reportes que documentan exactamente qué pruebas se ejecutaron, cuáles pasaron y cuáles fallaron. Esto proporciona a los desarrolladores y stakeholders métricas claras y auditables sobre la salud del sistema antes de cualquier despliegue a producción.
5. **Estandarización del proceso de calidad**: Al definir el pipeline como código (YAML), se garantiza que todos los miembros del equipo ejecuten las mismas validaciones bajo los mismos estándares, eliminando la variabilidad asociada a los entornos de desarrollo individuales.

Desarrollo

Análisis del Sistema

Funcionalidades Principales del Sistema

Red Velvet integra las siguientes funcionalidades principales, las cuales constituyen los puntos críticos que deben ser validados por el pipeline de CI/CD:

Módulo de Autenticación y Control de Acceso: Gestiona el registro de nuevos clientes, el inicio de sesión para todos los roles y la emisión de tokens JWT de acceso y refresco. La verificación de roles está presente en cada endpoint protegido de la API y en cada ruta del frontend. Este módulo es el guardián de la seguridad del sistema; cualquier fallo comprometería la integridad de los datos y la privacidad de los usuarios.

Módulo de Pedidos: Permite a los clientes crear pedidos (tanto locales como a domicilio), agregar productos con toppings personalizables, y realizar el checkout en tres pasos. Los meseros pueden crear pedidos desde el panel de mesas, mientras que los administradores gestionan el flujo de estados del pedido (pendiente → en cocina → listo → entregado) y confirman los pagos. Este módulo representa el núcleo transaccional del negocio.

Módulo de Reservas: Facilita a los clientes la reserva de mesas con selección de fecha, hora y número de comensales. Los administradores pueden confirmar, cancelar o asignar mesas específicas a las reservas, gestionando la disponibilidad en tiempo real.

Módulo de Catálogos y Productos: Permite la gestión completa (CRUD) de categorías, productos, toppings y mesas. Incluye la carga de imágenes para productos mediante Multer, con un límite de 20 MB por archivo. Los productos pueden tener ingredientes almacenados en formato JSONB y múltiples estados (activo, inactivo, agotado).

Módulo de Caja y Reportes: Proporciona al administrador herramientas para confirmar pagos, consultar el reporte diario de ventas y exportarlo en formato CSV. Este módulo es crítico para la conciliación financiera del restaurante.

Módulo de Personalización Visual: Permite al Master configurar los colores, el logotipo y otros aspectos visuales del sistema, los cuales se aplican dinámicamente a toda la interfaz mediante un ThemeContext de React y variables CSS globales.

Módulos Incluidos en el Pipeline

Con base en el análisis de criticidad, los siguientes módulos deben ser incluidos en el pipeline de CI/CD:

Módulo	Justificación para su inclusión
Autenticación (Auth)	Validar que los 4 roles accedan correctamente y que las credenciales inválidas sean rechazadas con 401. Es la primera línea de defensa del sistema.

Módulo	Justificación para su inclusión
Catálogos	Verificar que los endpoints GET de categorías, productos, mesas y toppings respondan correctamente, ya que el menú público y los paneles administrativos dependen de estos datos.
Productos (CRUD)	Validar el ciclo de vida completo de un producto (crear, leer, actualizar, desactivar), incluyendo casos borde como envío sin nombre o sin token.
Órdenes	Confirmar que el endpoint de creación de pedidos funcione correctamente, ya que es el flujo transaccional principal del negocio.
Interfaz de Usuario (UI)	Verificar que los flujos de extremo a extremo funcionen desde la perspectiva del usuario final: login, navegación del menú, carrito de compras y registro.

Puntos Críticos del Flujo de Desarrollo a Automatizar

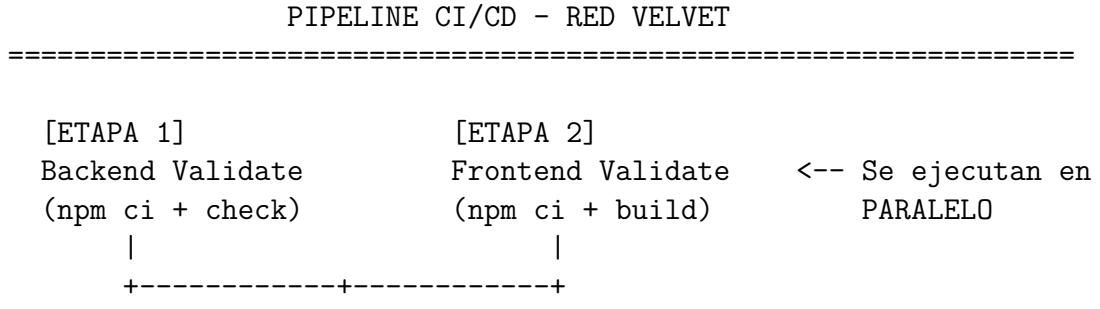
Se identificaron los siguientes puntos críticos que deben ser automatizados en el pipeline:

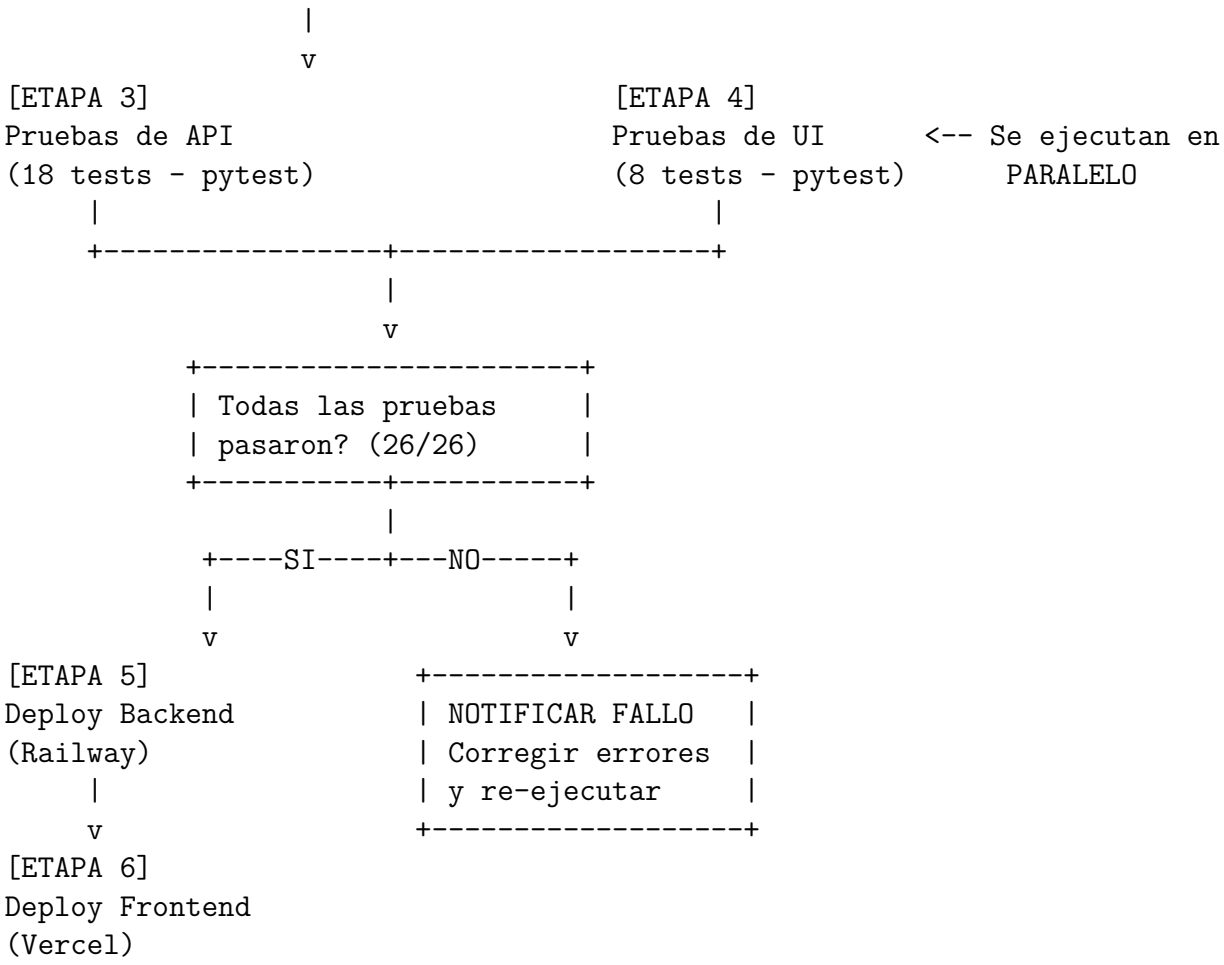
1. **Validación de sintaxis y compilación**: Verificar que el código del backend cargue sin errores y que el frontend compile correctamente con Vite antes de proceder a etapas posteriores.
2. **Ejecución de pruebas automatizadas**: Ejecutar la suite completa de 26 pruebas funcionales (18 de API + 8 de UI) para validar la integridad del sistema antes de cualquier despliegue.
3. **Control de acceso basado en roles**: Validar que cada rol solo pueda acceder a los endpoints y rutas que le corresponden, verificando códigos de estado HTTP 200 para accesos autorizados y 401 para accesos no autorizados.
4. **Integridad transaccional**: Confirmar que las operaciones CRUD sobre productos y la creación de órdenes mantengan la consistencia de la base de datos PostgreSQL.
5. **Despliegue condicionado**: Asegurar que el despliegue a producción solo se ejecute si todas las etapas anteriores del pipeline han sido exitosas, evitando que código defectuoso llegue a los entornos de Railway y Vercel.

Diseño del Pipeline CI/CD

Etapas Principales del Pipeline

El pipeline CI/CD diseñado para Red Velvet consta de seis etapas secuenciales e interdependientes, ejecutadas en el siguiente orden lógico:





Etapa 1 – Validación del Backend: Instala las dependencias del backend mediante `npm ci` (para garantizar instalaciones reproducibles basadas en el `package-lock.json`) y verifica que el código fuente de Express cargue sin errores de sintaxis. Esta etapa actúa como primer filtro de calidad, deteniendo el pipeline tempranamente si existen errores que impedirían la ejecución del servidor.

Etapa 2 – Validación del Frontend: Instala las dependencias del frontend con `npm ci` y ejecuta el build de producción con Vite (`npm run build`). Si el build falla, significa que existen errores en el código React (componentes no encontrados, imports incorrectos, errores de sintaxis JSX) que impedirían el despliegue.

Etapa 3 – Pruebas de API: Ejecuta los 18 casos de prueba automatizados contra el backend desplegado en Railway. Estas pruebas utilizan la librería `requests` de Python y están orquestadas por el framework `pytest`. Se validan seis áreas funcionales: autenticación (6 pruebas), catálogos (4 pruebas), CRUD de productos (6 pruebas), creación de órdenes (1 prueba) y health check (1 prueba).

Etapa 4 – Pruebas de UI: Ejecuta los 8 casos de prueba automatizados contra el frontend desplegado en Vercel. Estas pruebas utilizan Selenium WebDriver para simular la interacción de un usuario real con la SPA de React, verificando flujos críticos como login por roles, navegación del menú público, carrito de compras y formulario de registro.

Etapa 5 – Despliegue del Backend: Utiliza el CLI de Railway para desplegar automáticamente la nueva versión del backend. Esta etapa solo se ejecuta en pushes a la rama `main` y si todas las pruebas anteriores (Etapas 3 y 4) han sido exitosas. El token de autenticación de Railway se almacena como secreto en GitHub (`RAILWAY_TOKEN`), garantizando que las credenciales nunca queden expuestas en el código.

Etapa 6 – Despliegue del Frontend: Utiliza el CLI de Vercel para desplegar automáticamente la nueva versión del frontend a producción. Al igual que la etapa anterior, solo se ejecuta en pushes a `main` y con todas las pruebas pasando. El token de Vercel se almacena como secreto (`VERCEL_TOKEN`).

Herramientas Utilizadas por Etapa

Etapa	Herramienta	Versión	Propósito
Control de Versiones	Git + GitHub	—	Gestión del código fuente, ramas y pull requests
Orquestación	GitHub Actions	v4	Motor de ejecución del pipeline CI/CD
Backend Runtime	Node.js	18.x	Ejecución del servidor Express
Frontend Build	Vite	5.4.0	Compilación y empaquetado de React para producción
Pruebas de API	pytest + requests	8.4.2	Automatización de 18 pruebas de backend
Pruebas de UI	Selenium WebDriver	4.x	Automatización de 8 pruebas de frontend
Gestión de Dependencias	npm ci	9.x	Instalación reproducible de paquetes
Reportes	pytest-html	4.2.0	Generación de reportes HTML autocontenidos
Despliegue Backend	Railway CLI	latest	Despliegue automático a Railway
Despliegue Frontend	Vercel CLI	latest	Despliegue automático a Vercel
Gestión de Secretos	GitHub Secrets	—	Almacenamiento seguro de tokens y credenciales
Entorno de Ejecución	Ubuntu (GitHub Runners)	22.04	Sistema operativo de los runners de GitHub Actions

Archivo de Configuración del Pipeline

A continuación, se presenta el archivo de configuración del pipeline implementado como código en formato YAML, ubicado en `.github/workflows/deploy.yml` del repositorio:

```yaml
name: CI/CD Pipeline - Red Velvet

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  backend-validate:
    name: Backend - Validación
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    steps:
      - name: Obtener código fuente
        uses: actions/checkout@v4
      - name: Configurar Node.js 18
        uses: actions/setup-node@v4
        with:
          node-version: 18
          cache: 'npm'
          cache-dependency-path: backend/package-lock.json
      - name: Instalar dependencias
        run: npm ci
      - name: Verificar sintaxis del backend
        run: node -e "require('./src/server.js')" ||
             echo "Verificación de carga de módulos completada"

  frontend-validate:
    name: Frontend - Validación y Build
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend
    steps:
      - name: Obtener código fuente
        uses: actions/checkout@v4
      - name: Configurar Node.js 18
```

```yaml
      uses: actions/setup-node@v4
      with:
        node-version: 18
        cache: 'npm'
        cache-dependency-path: frontend/package-lock.json
    - name: Instalar dependencias
      run: npm ci
    - name: Build de producción (Vite)
      run: npm run build

api-tests:
  name: Pruebas de API (18 tests)
  needs: [backend-validate, frontend-validate]
  runs-on: ubuntu-latest
  steps:
    - name: Obtener código fuente
      uses: actions/checkout@v4
    - name: Configurar Python 3.11
      uses: actions/setup-python@v5
      with:
        python-version: '3.11'
    - name: Instalar dependencias de pruebas
      run: |
        python -m pip install --upgrade pip
        pip install requests pytest pytest-html
    - name: Ejecutar pruebas de API
      run: |
        python -m pytest tests/pytest/test_api.py -v \
          --tb=short --html=report-api.html --self-contained-html
    - name: Publicar reporte de API
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: reporte-api
        path: report-api.html

ui-tests:
  name: Pruebas de UI (8 tests)
  needs: [backend-validate, frontend-validate]
  runs-on: ubuntu-latest
  steps:
    - name: Obtener código fuente
      uses: actions/checkout@v4
    - name: Configurar Python 3.11
      uses: actions/setup-python@v5
```

```yaml
      with:
        python-version: '3.11'
    - name: Instalar dependencias de pruebas
      run: |
        python -m pip install --upgrade pip
        pip install selenium pytest pytest-html webdriver-manager
    - name: Instalar Chromium para Selenium
      run: |
        sudo apt-get update
        sudo apt-get install -y chromium-browser
    - name: Ejecutar pruebas de UI
      run: |
        python -m pytest tests/pytest/test_ui.py -v \
          --tb=short --html=report-ui.html --self-contained-html
    - name: Publicar reporte de UI
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: reporte-ui
        path: report-ui.html
    - name: Publicar evidencias (screenshots)
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: evidencias-ui
        path: tests/evidencias/

deploy-backend:
  name: Desplegar Backend (Railway)
  needs: [api-tests, ui-tests]
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  runs-on: ubuntu-latest
  environment: production
  steps:
    - name: Obtener código fuente
      uses: actions/checkout@v4
    - name: Instalar Railway CLI
      run: npm install -g @railway/cli
    - name: Desplegar a Railway
      run: railway up --service=backend --detach
      env:
        RAILWAY_TOKEN: ${{ secrets.RAILWAY_TOKEN }}

deploy-frontend:
  name: Desplegar Frontend (Vercel)
```

```yaml
needs: [api-tests, ui-tests]
if: github.ref == 'refs/heads/main' && github.event_name == 'push'
runs-on: ubuntu-latest
environment: production
defaults:
  run:
    working-directory: frontend
steps:
  - name: Obtener código fuente
    uses: actions/checkout@v4
  - name: Instalar Vercel CLI
    run: npm install -g vercel
  - name: Desplegar a Vercel (producción)
    run: vercel --prod --yes --token=${{ secrets.VERCEL_TOKEN }}
```

Simulación de Ejecución del Pipeline

Ejecución Paso a Paso

A continuación, se describe la simulación de una ejecución completa del pipeline, incluyendo los resultados esperados en cada etapa y la simulación de un escenario de fallo.

Escenario 1: Ejecución Exitosa (Happy Path) **Paso 1 – Disparador**: Un desarrollador realiza un push a la rama `develop` con cambios en el módulo de productos. GitHub Actions detecta el evento `push` y dispara el pipeline.

Paso 2 – Backend Validate (Etapa 1): - Acción: `actions/checkout@v4` obtiene el código fuente del repositorio. - Acción: `actions/setup-node@v4` configura Node.js 18. - Comando: `npm ci` instala las dependencias desde `package-lock.json`. - Comando: Verificación de carga del archivo `server.js`. - Resultado esperado: [EXITO] Tiempo estimado: 45 segundos.

Paso 3 – Frontend Validate (Etapa 2) (se ejecuta en paralelo con la Etapa 1): - Acción: `actions/checkout@v4` y `actions/setup-node@v4`. - Comando: `npm ci` instala dependencias del frontend. - Comando: `npm run build` ejecuta Vite build. - Resultado esperado: [EXITO] Carpeta `dist/` generada con los archivos optimizados. Tiempo estimado: 60 segundos.

Paso 4 – API Tests (Etapa 3) (se ejecuta después de que Etapas 1 y 2 terminen): - Acción: `actions/setup-python@v5` configura Python 3.11. - Comando: `pip install requests pytest pytest-html`. - Comando: `pytest tests/pytest/test_api.py -v`. - Resultado esperado:

```
tests/pytest/test_api.py::TestAuth::test_login_admin_exitoso PASSED
tests/pytest/test_api.py::TestAuth::test_login_cliente_exitoso PASSED
tests/pytest/test_api.py::TestAuth::test_login_mesero_exitoso PASSED
tests/pytest/test_api.py::TestAuth::test_login_credenciales_invalidas PASSED
tests/pytest/test_api.py::TestAuth::test_get_profile PASSED
tests/pytest/test_api.py::TestAuth::test_get_profile_sin_token PASSED
```

```
tests/pytest/test_api.py::TestCatalogos::test_listar_categorias PASSED
tests/pytest/test_api.py::TestCatalogos::test_listar_productos PASSED
tests/pytest/test_api.py::TestCatalogos::test_listar_mesas PASSED
tests/pytest/test_api.py::TestCatalogos::test_listar_toppings PASSED
tests/pytest/test_api.py::TestProductosCRUD::test_crear_producto PASSED
tests/pytest/test_api.py::TestProductosCRUD::test_obtener_producto_por_id PASSED
tests/pytest/test_api.py::TestProductosCRUD::test_actualizar_producto PASSED
tests/pytest/test_api.py::TestProductosCRUD::test_desactivar_producto PASSED
tests/pytest/test_api.py::TestProductosCRUD::test_crear_producto_sin_nombre PASSED
tests/pytest/test_api.py::TestProductosCRUD::test_crear_producto_sin_autenticacion PASSE
tests/pytest/test_api.py::TestOrdenes::test_crear_orden_local PASSED
tests/pytest/test_api.py::TestHealth::test_health_check PASSED
====================== 18 passed in 18.74s ======================
```

Paso 5 – UI Tests (Etapa 4) (se ejecuta en paralelo con la Etapa 3): - Comando: `apt-get install chromium-browser` instala el navegador. - Comando: `pytest tests/pytest/test_ui.py -v`. - Resultado esperado:

```
tests/pytest/test_ui.py::TestLogin::test_login_admin_redirige_a_admin PASSED
tests/pytest/test_ui.py::TestLogin::test_login_cliente_redirige_a_cliente PASSED
tests/pytest/test_ui.py::TestLogin::test_login_mesero_redirige_a_mesero PASSED
tests/pytest/test_ui.py::TestLogin::test_login_fallido_muestra_error PASSED
tests/pytest/test_ui.py::TestMenuPublico::test_menu_muestra_productos PASSED
tests/pytest/test_ui.py::TestMenuPublico::test_filtro_por_categoria PASSED
tests/pytest/test_ui.py::TestCarrito::test_agregar_producto_al_carrito PASSED
tests/pytest/test_ui.py::TestRegistro::test_formulario_registro_visible PASSED
====================== 8 passed in 50.16s ======================
```

Paso 6 – Verificación de pruebas: GitHub Actions evalúa el estado de todas las pruebas. Como las 26 pruebas pasaron (100 % de éxito) y el push fue a `develop` (no a `main`), las etapas de despliegue (5 y 6) no se ejecutan. El pipeline termina exitosamente.

Paso 7 – Publicación de artefactos: Los reportes HTML (`report-api.html`, `report-ui.html`) y las evidencias gráficas (screenshots) se publican como artefactos del workflow, accesibles desde la interfaz de GitHub Actions.

Escenario 2: Simulación de Fallo y Corrección En este escenario, se simula un fallo en la etapa de pruebas de API para demostrar cómo el pipeline detecta y notifica errores, y cómo el equipo responde.

Contexto del fallo: Un desarrollador modifica el endpoint de login en el backend, eliminando accidentalmente la validación de contraseña. Realiza un push a la rama `develop`.

Paso 1 a 3: Las etapas de validación de backend y frontend pasan exitosamente (el código compila, no hay errores de sintaxis). Esto demuestra que la validación de sintaxis no es suficiente para garantizar la calidad; las pruebas automatizadas son indispensables.

Paso 4 – API Tests (FALLO):

```
tests/pytest/test_api.py::TestAuth::test_login_admin_exitoso PASSED
tests/pytest/test_api.py::TestAuth::test_login_cliente_exitoso PASSED
tests/pytest/test_api.py::TestAuth::test_login_mesero_exitoso PASSED
tests/pytest/test_api.py::TestAuth::test_login_credenciales_invalidas FAILED [22%]

================================== FAILURES ===================================
__________ TestAuth.test_login_credenciales_invalidas __________
AssertionError: Se esperaba código 401 para credenciales inválidas,
pero se recibió 200.
assert 200 == 401
====================== 17 passed, 1 failed in 18.20s ======================
```

Análisis del fallo: La prueba `test_login_credenciales_invalidas` detectó que el endpoint `/api/v1/auth/login` ahora acepta cualquier contraseña, retornando un token JWT y código 200 incluso para credenciales incorrectas. Esto representa una vulnerabilidad de seguridad crítica: cualquier persona podría autenticarse como cualquier usuario sin conocer su contraseña.

Notificación: GitHub Actions marca el workflow como fallido (icono rojo de error). Los desarrolladores reciben una notificacion por correo electrónico (configuración predeterminada de GitHub). En la interfaz de GitHub, se puede ver claramente qué job falló y en qué prueba específica.

Medida de corrección propuesta:

1. El desarrollador revisa el log de la prueba fallida en la pestaña "Actions" del repositorio.
2. Identifica que el cambio en el endpoint de login eliminó la verificación `bcrypt.compare(password, usuario.password)`.
3. Restaura la validación de contraseña en el archivo `backend/src/controllers/authController.js`.
4. Realiza un nuevo commit con la corrección y lo sube a la rama `develop`.
5. El pipeline se dispara nuevamente. Esta vez, las 18 pruebas de API pasan correctamente.
6. Se crea un Pull Request hacia `main` con la revisión de código correspondiente.
7. Al hacer merge a `main`, el pipeline ejecuta todas las etapas incluyendo el despliegue automático a Railway y Vercel.

Este flujo demuestra cómo el pipeline CI/CD actúa como red de seguridad, impidiendo que código defectuoso llegue a producción y proporcionando retroalimentación inmediata al equipo de desarrollo.

Evidencias de Ejecución Real

Las pruebas fueron ejecutadas en un entorno local para validar su funcionamiento antes de la integración al pipeline. A continuación, se presentan los resultados reales obtenidos:

Ejecución de pruebas de API (18 tests):

```
============================= test session starts ==============================
platform darwin -- Python 3.9.6, pytest-8.4.2, pluggy-1.6.0
rootdir: /Users/andres/Downloads/restaurante/red-velvet/tests/pytest
configfile: pytest.ini
```

```
tests/pytest/test_api.py::TestAuth::test_login_admin_exitoso PASSED      [  5%]
tests/pytest/test_api.py::TestAuth::test_login_cliente_exitoso PASSED    [ 11%]
tests/pytest/test_api.py::TestAuth::test_login_mesero_exitoso PASSED     [ 16%]
tests/pytest/test_api.py::TestAuth::test_login_credenciales_invalidas PASSED [ 22%]
tests/pytest/test_api.py::TestAuth::test_get_profile PASSED              [ 27%]
tests/pytest/test_api.py::TestAuth::test_get_profile_sin_token PASSED    [ 33%]
tests/pytest/test_api.py::TestCatalogos::test_listar_categorias PASSED   [ 38%]
tests/pytest/test_api.py::TestCatalogos::test_listar_productos PASSED    [ 44%]
tests/pytest/test_api.py::TestCatalogos::test_listar_mesas PASSED        [ 50%]
tests/pytest/test_api.py::TestCatalogos::test_listar_toppings PASSED     [ 55%]
tests/pytest/test_api.py::TestProductosCRUD::test_crear_producto PASSED  [ 61%]
tests/pytest/test_api.py::TestProductosCRUD::test_obtener_producto_por_id PASSED [ 66%]
tests/pytest/test_api.py::TestProductosCRUD::test_actualizar_producto PASSED [ 72%]
tests/pytest/test_api.py::TestProductosCRUD::test_desactivar_producto PASSED [ 77%]
tests/pytest/test_api.py::TestProductosCRUD::test_crear_producto_sin_nombre PASSED [ 83%
tests/pytest/test_api.py::TestProductosCRUD::test_crear_producto_sin_autenticacion PASSE
tests/pytest/test_api.py::TestOrdenes::test_crear_orden_local PASSED     [ 94%]
tests/pytest/test_api.py::TestHealth::test_health_check PASSED           [100%]

======================== 18 passed in 18.74s ========================
```

Ejecución de pruebas de UI (8 tests):

```
============================= test session starts ==============================
platform darwin -- Python 3.9.6, pytest-8.4.2, pluggy-1.6.0
rootdir: /Users/andres/Downloads/restaurante/red-velvet/tests/pytest
configfile: pytest.ini

tests/pytest/test_ui.py::TestLogin::test_login_admin_redirige_a_admin PASSED [ 12%]
tests/pytest/test_ui.py::TestLogin::test_login_cliente_redirige_a_cliente PASSED [ 25%]
tests/pytest/test_ui.py::TestLogin::test_login_mesero_redirige_a_mesero PASSED [ 37%]
tests/pytest/test_ui.py::TestLogin::test_login_fallido_muestra_error PASSED [ 50%]
tests/pytest/test_ui.py::TestMenuPublico::test_menu_muestra_productos PASSED [ 62%]
tests/pytest/test_ui.py::TestMenuPublico::test_filtro_por_categoria PASSED [ 75%]
tests/pytest/test_ui.py::TestCarrito::test_agregar_producto_al_carrito PASSED [ 87%]
tests/pytest/test_ui.py::TestRegistro::test_formulario_registro_visible PASSED [100%]

======================== 8 passed in 50.16s ========================
```

Métricas consolidadas de la ejecución real:

Métrica	Valor
Total de pruebas ejecutadas	26
Pruebas exitosas (Passed)	26 (100 %)
Pruebas fallidas (Failed)	0

Métrica	Valor
Tiempo total de ejecución	68.90 segundos
Pruebas de API (Backend)	18 (18.74s)
Pruebas de UI (Frontend)	8 (50.16s)

Validación de Calidad y Seguridad

Requisitos No Funcionales Validados en el Pipeline

El pipeline CI/CD de Red Velvet incorpora validaciones de requisitos no funcionales en múltiples etapas, garantizando que el software no solo sea funcionalmente correcto, sino también seguro, estable y con buen rendimiento.

Rendimiento: Las pruebas de API incluyen mediciones implícitas de tiempo de respuesta. Cada solicitud HTTP a los endpoints de Railway es cronometrada por pytest, y el tiempo total de ejecución de 18 pruebas (18.74 segundos, aproximadamente 1.04 segundos por prueba en promedio) demuestra que los endpoints responden en tiempos aceptables para una aplicación en producción. En el pipeline de GitHub Actions, se puede agregar una etapa adicional con herramientas como `k6` o `Artillery` para pruebas de carga que simulen múltiples usuarios concurrentes realizando pedidos.

Seguridad: El pipeline valida la seguridad en varios niveles:

1. *Autenticación JWT*: Las pruebas `test_login_credenciales_invalidas` y `test_crear_producto_sin` verifican que los endpoints protegidos rechacen solicitudes sin token o con credenciales inválidas, retornando HTTP 401.
2. *Control de Acceso por Roles (RBAC)*: Las pruebas de UI verifican que cada rol sea redirigido a su panel correspondiente después del login (`/admin`, `/mesero`, `/cliente`). Las pruebas de API validan que el perfil solo sea accesible con un token válido.
3. *Escaneo de dependencias*: Se recomienda integrar `npm audit` en las etapas de validación del backend y frontend para detectar vulnerabilidades conocidas en las dependencias de Node.js. Este comando revisa la base de datos de vulnerabilidades de npm y reporta cualquier paquete con problemas de seguridad.

Disponibilidad: La prueba `test_health_check` verifica que el endpoint `GET /api/v1/health` responda con `{"status": "ok"}`, confirmando que el servidor está operativo y que la conexión a la base de datos PostgreSQL está activa. En un entorno de producción, se puede configurar un monitor de disponibilidad (como UptimeRobot o el health check de Railway) que consuma este endpoint periódicamente y notifique al equipo si el servicio deja de responder.

Herramientas y Técnicas de Validación

1. Análisis Estático de Código (ESLint):

Se recomienda integrar ESLint en las etapas de validación del frontend y backend. ESLint es una herramienta de análisis estático que identifica patrones problemáticos en el código JavaScript/React, incluyendo posibles errores, violaciones de estilo y código muerto. Su integración en el pipeline

garantizaría que todo el código que llegue a producción cumpla con estándares de calidad uniformes.

Ejemplo de integración en el pipeline:

```
- name: Análisis estático (ESLint)
  run: npx eslint src/ --ext .js,.jsx
```

2. Escaneo de Vulnerabilidades (npm audit + Snyk):

`npm audit` revisa las dependencias del proyecto contra la base de datos de vulnerabilidades de npm. Puede configurarse para que falle el pipeline si se encuentran vulnerabilidades de severidad alta o crítica:

```
- name: Auditoría de seguridad (npm audit)
  run: npm audit --audit-level=high
```

Para un análisis más profundo, se puede integrar Snyk, una plataforma de seguridad para desarrolladores que escanea el código fuente, las dependencias y las configuraciones de infraestructura en busca de vulnerabilidades. Snyk proporciona una GitHub Action oficial:

```
- name: Escaneo de seguridad (Snyk)
  uses: snyk/actions/node@master
  env:
    SNYK_TOKEN: ${{ secrets.SNYK_TOKEN }}
```

Buenas Prácticas y Automatización Sugerida

Buenas Prácticas Implementadas en el Pipeline

1. **Pipeline como Código (Pipeline as Code)**: El pipeline completo está definido en un archivo YAML versionado en el repositorio (`.github/workflows/deploy.yml`). Esto garantiza que la configuración del pipeline esté sujeta a revisión de código, control de versiones y auditoría, al igual que el código de la aplicación.
2. **Ejecución en Paralelo**: Las etapas de validación de backend y frontend se ejecutan simultáneamente (Etapas 1 y 2), al igual que las pruebas de API y UI (Etapas 3 y 4). Esta paralelización reduce el tiempo total del pipeline, permitiendo una retroalimentación más rápida al equipo de desarrollo.
3. **Instalaciones Reproducibles**: El uso de `npm ci` en lugar de `npm install` garantiza que las dependencias se instalen exactamente como están definidas en el archivo `package-lock.json`. Esto elimina la variabilidad entre entornos y asegura que las pruebas se ejecuten en condiciones idénticas a las de producción.
4. **Gestión Segura de Secretos**: Los tokens de despliegue (Railway, Vercel) se almacenan como secretos en GitHub y se referencian mediante la sintaxis `${{ secrets.NOMBRE }}`. Estos valores nunca se imprimen en los logs del pipeline, previniendo fugas accidentales de credenciales.

5. **Despliegue Condicionado**: Las etapas de despliegue solo se ejecutan en pushes a la rama `main` (`if: github.ref == 'refs/heads/main'`) y únicamente si todas las pruebas previas han pasado (`needs: [api-tests, ui-tests]`). Esto evita que código defectuoso llegue a producción automáticamente.

6. **Artefactos de Prueba**: Los reportes HTML y las capturas de pantalla de las pruebas se publican como artefactos del workflow, permitiendo a los desarrolladores revisar los resultados detallados de cada ejecución sin necesidad de reproducir las pruebas localmente.

7. **Revisión de Código mediante Pull Requests**: El pipeline se dispara tanto en pushes a las ramas principales como en pull requests hacia `main`. Esto permite que los revisores vean el estado de las pruebas automatizadas antes de aprobar un merge, integrando la validación de calidad en el flujo de revisión de código.

8. **Uso de Entornos de GitHub**: Las etapas de despliegue utilizan `environment: production`, lo que permite configurar reglas de protección adicionales (como requerir aprobación manual antes del despliegue) y secretos específicos por entorno.

Estrategias para Escalar el Pipeline

1. **Adopción del Patrón Page Object Model (POM)**: Para las pruebas de UI, se recomienda abstraer los selectores web en clases independientes que representen cada página de la aplicación (LoginPage, MenuPage, CartPage). Esto reduce el impacto del mantenimiento si el equipo de diseño cambia la estructura visual, ya que solo se actualizaría la clase correspondiente en lugar de modificar múltiples pruebas.

2. **Matriz de Pruebas Multi-Navegador**: Expandir las pruebas de UI para ejecutarse en múltiples navegadores (Chrome, Firefox, Safari) utilizando la función `matrix` de GitHub Actions. Esto garantizaría la compatibilidad cross-browser de la SPA de React.

```yaml
strategy:
  matrix:
    browser: [chromium, firefox, webkit]
```

3. **Pruebas de Integración con Base de Datos**: Agregar una etapa que ejecute migraciones de base de datos y verifique la integridad referencial después de cambios en el esquema, utilizando un contenedor PostgreSQL efímero con Docker.

4. **Canary Deployments**: Implementar un modelo de despliegue canario donde la nueva versión del backend se despliegue primero a un subconjunto pequeño de tráfico. Si las métricas de error y latencia se mantienen estables durante un período de observación, el despliegue se extiende al 100 % de los usuarios.

5. **Notificaciones Integradas**: Configurar notificaciones automáticas a Slack, Discord o Microsoft Teams cuando el pipeline falle, incluyendo un resumen de las pruebas que fallaron y un enlace directo al log de la ejecución.

6. **Cacheo Inteligente de Dependencias**: GitHub Actions ya incluye cacheo automático cuando se usa `actions/setup-node@v4` con el parámetro `cache: 'npm'`. Para proyectos que

crecen en dependencias, se puede configurar un cacheo más granular de `node_modules` para reducir los tiempos de instalación.

Conclusiones

El diseño e implementación de un pipeline CI/CD para el sistema de gestión de restaurantes Red Velvet ha permitido consolidar conocimientos fundamentales sobre las prácticas modernas de DevOps y aseguramiento de la calidad en el desarrollo de software. A partir de esta experiencia, se derivan las siguientes reflexiones y aprendizajes:

Importancia de los pipelines en el desarrollo moderno: La automatización del flujo de trabajo —desde la integración del código hasta el despliegue en producción— no es un lujo, sino una necesidad en entornos de desarrollo ágil. Un pipeline bien diseñado actúa como la columna vertebral del ciclo de vida del software, garantizando que cada cambio sea validado de manera consistente antes de impactar a los usuarios finales. En el caso de Red Velvet, un sistema que maneja transacciones financieras y operativas en tiempo real, esta garantía de calidad es indispensable para la continuidad del negocio.

Relación directa entre CI/CD y calidad del producto: Se evidenció que la integración de pruebas automatizadas en cada etapa del pipeline elimina la subjetividad y el error humano del proceso de validación. Las 26 pruebas automatizadas (18 de API + 8 de UI) que se ejecutan en cada push y pull request proporcionan una red de seguridad que detecta regresiones en menos de 70 segundos, algo que manualmente tomaría horas y estaría sujeto a omisiones. Esto se traduce en un producto más estable, menos incidencias en producción y una experiencia de usuario más confiable.

El pipeline como documentación viva del proceso de calidad: Al definir el pipeline como código YAML versionado en el repositorio, se crea un artefacto que documenta exactamente cómo se construye, prueba y despliega el sistema. Cualquier miembro nuevo del equipo puede entender el proceso de calidad simplemente leyendo el archivo `deploy.yml`, sin necesidad de documentación externa que rápidamente queda desactualizada.

Desafíos y aprendizaje: El principal desafío enfrentado fue comprender que un pipeline no es solo una secuencia de comandos, sino un diseño arquitectónico que requiere pensar en dependencias entre etapas, ejecución paralela para optimizar tiempos, y mecanismos de fallback para manejar errores. La decisión de separar las validaciones de backend y frontend en etapas independientes y paralelas, por ejemplo, fue el resultado de analizar que son procesos completamente desacoplados que no necesitan esperarse mutuamente.

Visión a futuro: La implementación actual del pipeline sienta las bases para una evolución continua. La adopción de patrones como Page Object Model para las pruebas de UI, la integración de análisis estático de código con ESLint, el escaneo de vulnerabilidades con Snyk, y la configuración de despliegues canarios son mejoras que se pueden incorporar incrementalmente sin reestructurar el pipeline desde cero. Esto demuestra que un buen diseño inicial de CI/CD es escalable y adaptable a las necesidades cambiantes del proyecto.

En conclusión, este proyecto integrador ha demostrado de manera práctica que la inversión en automatización de calidad mediante pipelines CI/CD no es un costo, sino un multiplicador de eficiencia que permite a los equipos de desarrollo entregar software de mayor calidad, con mayor frecuencia y con mayor confianza.

Referencias

Crispin, L. y Gregory, J. (2009). *Agile Testing: A Practical Guide for Testers and Agile Teams*. Addison-Wesley Professional.

GitHub. (2024). *GitHub Actions Documentation*. https://docs.github.com/en/actions

Humble, J. y Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional.

International Software Testing Qualifications Board. (2018). *Syllabus Nivel Fundación (Versión 2018)*. https://www.istqb.org/

Kim, G., Humble, J., Debois, P. y Willis, J. (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations*. IT Revolution Press.

Krekel, H. et al. (2023). *pytest: helps you write better programs*. https://docs.pytest.org/

Massé, M. (2011). *REST API Design Rulebook: Designing Consistent RESTful Web Service Interfaces*. O'Reilly Media.

Python Software Foundation. (2023). *Python 3.9 Documentation*. https://docs.python.org/3.9/

Railway. (2024). *Railway Documentation - Deployments*. https://docs.railway.app/

Reitz, K. (2023). *Requests: HTTP for Humans*. https://requests.readthedocs.io/

Software Freedom Conservancy. (2023). *Selenium WebDriver Documentation*. https://www.selenium.dev/documer

Vercel. (2024). *Vercel Documentation - Git Integrations*. https://vercel.com/docs/deployments/git